



Centrifuge V3.3 Audit Report

Prepared by [Burra Security](#)

Version 1.0

Researchers

Alex Filippov

Taridoku

fuzious

typicalHuman

Mahdi Karimi

August 13, 2026

Contents

1	About Burra Security	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
5	Audit Scope	3
6	Executive Summary	4
7	Findings	7
7.1	Medium Risk	7
7.1.1	One faulty adapter halts all outbound dispatch	7
7.1.2	A manager can set an arbitrary transfer hook instantly on first share-class deployment	7
7.1.3	Large NAV swing persistently freezes the accounting sync pipeline	8
7.1.4	Sub-rateMax bridge transfers blocked by an exhausted window cannot be individually authorized	8
7.1.5	A nonzero-delay StdManifest blocks OracleValuation feeder updates	10
7.1.6	Global-group votes leak into a pool's group, letting one relayer in each take over the pool	10
7.1.7	Message types were renumbered, so messages sent before the update are read as the wrong type after it	12
7.1.8	Stale adapter updates can roll back active adapter sets	13
7.1.9	Delayed Emergency Freezes Let Shareholders Evade Restrictions	14
7.1.10	Circuit Breaker Replacement Resets Active Rate-Limit Budget	15
7.1.11	Stale Oracle Retry Can Roll Back a Newer Price	15
7.1.12	Ceil-Rounded Deviation Band Permits Zero BRM Price	16
7.1.13	Reordered SetPoolAdapters messages can desynchronize adapter configurations	17
7.2	Low Risk	18
7.2.1	Fixed force-cancel opt-in lets a manager repeatedly wipe an investor's fresh requests	18
7.2.2	The sentinel-removal guard blocks only self-removal	18
7.2.3	Hyperlane messages to Monad can be underfunded because the adapter uses a chain-independent receive reserve	19
7.2.4	Hub.setAdapters refund callback reentrancy can desync hub and spoke adapters	20
7.2.5	Hub.cancelAuthorization bypasses the manifest allowlist, leaking a governance-DoS primitive to confined managers	21
7.2.6	A stale cross-chain manager update can restore revoked remote authority	22
7.2.7	HyperlaneAdapter never checks that a security module (ISM) is set, so it can silently fall back to the Mailbox default	22
7.2.8	Circuit-breaker authorizations can be reused across destination chains	23
7.3	Informational	24
7.3.1	Superseded adapter sessions authenticate indefinitely	24
7.3.2	MultiAdapter.setAdapters accepts threshold == 0, silently disabling inbound adapter consensus	24
7.3.3	Reinstalling a prior manifest revives its unexpired authorizations without a fresh veto window	25
7.3.4	StdManifest does not check the timelock values, so a bad config can remove the delay that protects setManifest	25
7.3.5	Adapter Failover Timelock May Expire Before Hub Veto Arrives	26
7.3.6	Initial Managers Can Bypass Hub Manifest Protections	26
7.3.7	ISO Holdings Accept Updates From Unauthorized Sources	27
7.3.8	Stale Steward Updates Can Restore Revoked Failover Authority	28
7.3.9	Wrapped MultiAdapter session ids can revive orphaned votes	28
7.3.10	Three Hub accounting selectors are unavailable under onchainAccounting	29
7.3.11	HubRegistry.setBridgingHook does not check that the pool exists	29
7.3.12	Some Hub and HubHandler dependencies cannot be updated	30

7.3.13 MessageLib.messageLength accepts the invalid message type 30

7.3.14 OpsGuardian.wire permits the local chain 30

7.3.15 StandbyAdapter does not emit the underlying adapterData 31

7.3.16 ManagerCall NatSpec names the wrong trusted-update target 31

7.3.17 Supervisor uses stale trusted-call terminology 31

7.3.18 INAVManager declares two unused errors 32

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Centrifuge is a Hub-and-Spoke protocol for onchain asset management, where a pool's Hub chain drives its Spoke deployments over a cross-chain messaging layer. The in-scope contracts cover this release's two additions: a manifest-based permission layer that constrains what a pool manager can do on the Hub, and a reworked messaging stack with per-pool adapter sets and bridge failover.

Every guarded Hub call passes through the pool's manifest. StdManifest classifies it — in-policy calls execute immediately, while out-of-policy calls must be pre-authorized in HubRegistry's shared authorization ledger, age past a delay, and be consumed within an expiry window or fail closed. Supervisor registers pool sentinels whose sole power is vetoing a pending authorization during that window, and replacing the manifest itself carries a longer escalation delay.

Hub.managerCall routes through Envoy, a stateless dispatcher that gives manager contracts a stable `msg.sender` anchor across Hub redeployments. Behind it sit OracleValuation for trusted per-asset price feeds, NAVManager for onchain NAV accounting, BatchRequestManager for epoch-batched deposit and redeem fulfillment, and BridgeCircuitBreaker, which pauses and rate-limits cross-chain share transfers through the IBridgingHook pre-transfer hook.

MultiAdapter keys adapter sets per pool and destination chain, versions them by session, and clears inbound messages by M-of-N quorum. A new HyperlaneAdapter joins the Axelar, Chainlink, and LayerZero routes, while StandbyAdapter fills a quorum slot for the cost of one storage write and lets anyone forward a recorded payload through its underlying adapter when a live route stalls. AdapterFailover lets a per-pool steward install a replacement adapter set behind a timelock, vetoable by the Hub while the existing adapters still work.

5 Audit Scope

The scope of the audit is the diff between commit `24f8b445086fad4bce080272f77878dc650caba7` and commit `23b3e9666d60c2d014092e4b60e4360017e23509` for all the files in the:

```
src/  
├── adapters/
```

```

├── HyperlaneAdapter.sol
├── StandbyAdapter.sol
├── admin/
│   ├── GasService.sol
│   ├── OpsGuardian.sol
│   └── ProtocolGuardian.sol
├── core/
│   ├── hub/
│   │   ├── Hub.sol
│   │   ├── HubHandler.sol
│   │   └── HubRegistry.sol
│   ├── messaging/
│   │   ├── libraries/
│   │   │   └── MessageLib.sol
│   │   ├── Gateway.sol
│   │   ├── MessageDispatcher.sol
│   │   ├── MessageProcessor.sol
│   │   └── MultiAdapter.sol
│   └── utils/
│       ├── BatchedMulticall.sol
│       ├── ContractUpdateLib.sol
│       ├── ContractUpdaterForwarder.sol
│       └── Envoy.sol
├── hooks/
│   ├── accounting/
│   │   └── NAVManager.sol
│   └── bridge/
│       └── BridgeCircuitBreaker.sol
├── managers/
│   ├── adapters/
│   │   └── AdapterFailover.sol
│   └── hub/
│       └── Supervisor.sol
├── manifests/
│   └── StdManifest.sol
├── misc/
│   ├── libraries/
│   │   ├── ArrayLib.sol
│   │   ├── BytesLib.sol
│   │   └── MathLib.sol
│   └── types/
│       └── D18.sol
├── valuations/
│   └── OracleValuation.sol
└── vaults/
    └── BatchRequestManager.sol

```

6 Executive Summary

Over the course of 8 days, the Burra Security team conducted an audit on the [Centrifuge V3.3](#) smart contracts provided by [Centrifuge](#). In this period, a total of 39 issues were found.

Summary

Project Name	Centrifuge V3.3
Repository	protocol-internal
Commit	23b3e9666d60...
Audit Timeline	Jul 6th - Jul 15th
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	13
Low Risk	8
Informational	18
Gas Optimizations	0
Total Issues	39

Summary of Findings

[M-01] One faulty adapter halts all outbound dispatch	Acknowledged
[M-02] A manager can set an arbitrary transfer hook instantly on first share-class deployment	Resolved
[M-03] Large NAV swing persistently freezes the accounting sync pipeline	Resolved
[M-04] Sub-rateMax bridge transfers blocked by an exhausted window cannot be individually authorized	Resolved
[M-05] A nonzero-delay StdManifest blocks OracleValuation feeder updates	Resolved
[M-06] Global-group votes leak into a pool's group, letting one relayer in each take over the pool	Resolved
[M-07] Message types were renumbered, so messages sent before the update are read as the wrong type after it	Resolved
[M-08] Stale adapter updates can roll back active adapter sets	Resolved
[M-09] Delayed Emergency Freezes Let Shareholders Evade Restrictions	Resolved
[M-10] Circuit Breaker Replacement Resets Active Rate-Limit Budget	Acknowledged
[M-11] Stale Oracle Retry Can Roll Back a Newer Price	Resolved
[M-12] Ceil-Rounded Deviation Band Permits Zero BRM Price	Resolved
[M-13] Reordered SetPoolAdapters messages can desynchronize adapter configurations	Resolved
[L-1] Fixed force-cancel opt-in lets a manager repeatedly wipe an investor's fresh requests	Acknowledged

[L-2] The sentinel-removal guard blocks only self-removal	Acknowledged
[L-3] Hyperlane messages to Monad can be underfunded because the adapter uses a chain-independent receive reserve	Resolved
[L-4] Hub.setAdapters refund callback reentrancy can desync hub and spoke adapters	Resolved
[L-5] Hub.cancelAuthorization bypasses the manifest allowlist, leaking a governance-DoS primitive to confined managers	Resolved
[L-6] A stale cross-chain manager update can restore revoked remote authority	Acknowledged
[L-7] HyperlaneAdapter never checks that a security module (ISM) is set, so it can silently fall back to the Mailbox default	Resolved
[L-8] Circuit-breaker authorizations can be reused across destination chains	Resolved
[I-01] Superseded adapter sessions authenticate indefinitely	Resolved
[I-02] MultiAdapter.setAdapters accepts threshold == 0, silently disabling inbound adapter consensus	Resolved
[I-03] Reinstalling a prior manifest revives its unexpired authorizations without a fresh veto window	Resolved
[I-04] StdManifest does not check the timelock values, so a bad config can remove the delay that protects setManifest	Resolved
[I-05] Adapter Failover Timelock May Expire Before Hub Veto Arrives	Acknowledged
[I-06] Initial Managers Can Bypass Hub Manifest Protections	Acknowledged
[I-07] ISO Holdings Accept Updates From Unauthorized Sources	Resolved
[I-08] Stale Steward Updates Can Restore Revoked Failover Authority	Acknowledged
[I-09] Wrapped MultiAdapter session ids can revive orphaned votes	Resolved
[I-10] Three Hub accounting selectors are unavailable under onchainAccounting	Resolved
[I-11] HubRegistry.setBridgingHook does not check that the pool exists	Resolved
[I-12] Some Hub and HubHandler dependencies cannot be updated	Resolved
[I-13] MessageLib.messageLength accepts the invalid message type	Resolved
[I-14] OpsGuardian.wire permits the local chain	Resolved
[I-15] StandbyAdapter does not emit the underlying adapterData	Resolved
[I-16] ManagerCall NatSpec names the wrong trusted-update target	Resolved
[I-17] Supervisor uses stale trusted-call terminology	Resolved
[I-18] INAVManager declares two unused errors	Resolved

7 Findings

7.1 Medium Risk

7.1.1 One faulty adapter halts all outbound dispatch

Target: MultiAdapter.sol, Gateway.sol

Severity:

- Impact: Medium
- Likelihood: Medium

Description: MultiAdapter.send implements outbound redundancy as "send through every configured adapter" rather than "send through enough adapters to satisfy the threshold." It loops over the full active adapter list and, for each entry, _sendToAdapter calls adapter.estimate then adapter.send, bubbling any revert. The public estimate path has the same all adapters loop, and Gateway._send depends on that estimate before it ever calls a single adapter.send. So one configured adapter that reverts on estimate or send vetoes the entire outbound message, even when the pool is configured 2-of-3 and the other two adapters are healthy. Remediation is also blocked: Hub.setAdapters sends the remote SetPoolAdapters over the current set before applying the replacement, so the same reverting adapter blocks the clean rotation that would remove it, forcing the operator onto blockSession or a Root spell. It also prevents a configured StandbyAdapter from recording its StandbySend, since its zero-cost send is reached only if the earlier adapters in the loop do not revert.

Recommended Mitigation: Attempt each adapter independently, emit a per adapter failure event, and require only enough successful sends or standby records to make eventual threshold execution possible.

Centrifuge: We assume an adapter implementations should never revert in send/estimate. If for some reason, the adapter reverts, the pool manager can set an adapter manager in multiAdapter to call multiAdapter.setAdapters() and set new adapters freely without being blocked by the old breaking ones.

BurraSec: Acknowledged

7.1.2 A manager can set an arbitrary transfer hook instantly on first share-class deployment

Target: Hub.sol, StdManifest.sol, Spoke.sol

Severity:

- Impact: High
- Likelihood: Low

Description: The manifest guards changing a share token's transfer hook but not choosing it on first deployment. Hub.updateShareHook, which repoints the hook on an existing token, is classified out-of-policy delay (timelocked and sentinel-vetoable). But Hub.notifyShareClass, which deploys the share token on a target chain and sets its initial hook, is classified in policy (instant, no authorization, no veto window), on the premise that notify messages merely re-push already-committed state. That premise fails for the hook: notifyShareClass takes hook as a caller-supplied bytes32 argument passed straight through. The hub persists no canonical transfer hook per share class (addShareClass stores only name, symbol, salt), so the hook has no committed source, it is fresh unvalidated calldata on every call. So a pool manager can, for any chain where the share class is not yet deployed, call notifyShareClass with an attacker-chosen hook and have it take effect instantly.

Recommended Mitigation: Consider moving notifyShareClass into the delay group alongside updateShareHook and addShareClass, so the initial cross-chain hook install carries the same timelock and sentinel veto window as every other hook change.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/329>

BurraSec: Looks good, notifyShareClass now out of policy

7.1.3 Large NAV swing persistently freezes the accounting sync pipeline

Target: StdManifest.sol, SimplePriceManager.sol, Gateway.sol

Severity:

- Impact: High
- Likelihood: Low

Description: In onchainAccounting mode, an inbound accounting message eventually calls SimplePriceManager.onUpdate, which computes a new share price and calls Hub.updateSharePrice with computedAt = uint64(block.timestamp). If a legitimate NAV move reaches or exceeds maxAbsolutePriceDelta, StdManifest classifies the update as out of policy and requires a matured exact-calldata authorization. That recovery path is not practically usable for the automated update because HubRegistry keys the authorization by the full calldata, including the execution-time computedAt value and the price recomputed during processing. Without a matching authorization, the price update reverts; this unwinds the holding update and Holdings.setSnapshot nonce increment, while Gateway records the accounting message as failed. Later messages for the same (poolId, scId, centrifugeId) then fail with InvalidNonce behind the stuck message. Because the absolute-delta check is time-independent, waiting and calling Gateway.retry does not clear the blockage. Recovery requires coordinated privileged intervention, such as replacing the manifest or temporarily disabling the snapshot hook, leaving the affected accounting stream unavailable in the meantime.

Recommended Mitigation: Decouple accounting progress from acceptance of the derived share price. An out-of-policy automated price update should be rejected and surfaced through an explicit event without reverting the holding update or snapshot nonce advancement.

Centrifuge: Fixed indirectly in [centrifuge/protocol-internal#283](#), allowing the exact automated price update to be pre-authorized and retried after the manifest delay.

BurraSec: Fix looks good. Added a timestamp-free overload which stabilizes the authorization ID, while execution remains restricted to the configured SimplePriceManager. A failed accounting message can now be authorized and retried, allowing the snapshot nonce stream to resume.

7.1.4 Sub-rateMax bridge transfers blocked by an exhausted window cannot be individually authorized

Target: BridgeCircuitBreaker.sol, CircuitBreakerGuard.sol

Severity:

- Impact: Low
- Likelihood: High

Description: BridgeCircuitBreaker only consults manager authorizations when a transfer's individual amount exceeds rateMax:

```
if (p.amount > l.rateMax) {
    bytes32 key = _authKey(p.poolId, p.scId, p.originCentrifugeId, p.sender, p.receiver, p.amount);
    require(authorizations[key] > 0, TransferNotAuthorized());
    authorizations[key]--;
    return;
}

circuitBreakerGuard.tally(
    keccak256(abi.encode(p.poolId, p.scId, p.originCentrifugeId)), p.amount, l.rateMax, l.rateWindow
);
```

For any transfer with p.amount <= rateMax, the hook goes straight to tally(), which accumulates all transfers in the current window and reverts when the running total exceeds max:

```
require(newTotal <= max, ExceedsCumulativeLimit(key, amount, max, window));
```

A transfer that is individually below `rateMax` but exceeds the window's remaining capacity therefore reverts, and `AuthorizeTransfer` cannot rescue it: the authorizations mapping is never read on that path. A manager can authorize transfers larger than `rateMax`, but cannot approve a specific below-`rateMax` transfer that is blocked only because earlier transfers already consumed the window.

The only ways to let the transfer through are to wait for the rolling window to elapse or to raise `rateMax`, which relaxes the limit for the entire `(poolId, scId, originCentrifugeId)` bucket rather than approving one transfer.

Failure Scenario:

1. A bucket has `rateMax = 1000`, and earlier transfers in the same `(poolId, scId, originCentrifugeId)` window consume 900.
2. A manager bridges a legitimate 200 shares; the source-side Spoke burns the shares and sends the initiate-transfer message.
3. The amount is below `rateMax`, so the hook skips the authorization check and calls `tally()`, which computes $900 + 200$ and reverts with `ExceedsCumulativeLimit`.
4. A manager sends `AuthorizeTransfer` for that exact transfer, but a retry follows the same `tally()` path and reverts again.

Impact: A below-`rateMax` transfer blocked by the window cannot be approved individually, even by a manager. The only ways to release it are to wait for the window to reset or raise `rateMax`, which weakens the limit for the whole bucket.

Recommended Mitigation: Check whether the transfer exceeds the remaining cumulative allowance rather than only checking whether `p.amount > rateMax`.

Conceptually, the condition should be:

```
if (currentWindowTotal + p.amount > l.rateMax) {
    bytes32 key =
        _authKey(p.poolId, p.scId, p.originCentrifugeId, p.sender, p.receiver, p.amount);

    require(authorizations[key] > 0, TransferNotAuthorized());
    authorizations[key]--;
    return;
}

circuitBreakerGuard.tally(
    keccak256(abi.encode(p.poolId, p.scId, p.originCentrifugeId)),
    p.amount,
    l.rateMax,
    l.rateWindow
);
```

This may require `CircuitBreakerGuard` to expose the amount already consumed or the remaining allowance for the current rolling window.

Alternatively, an exact authorization can be checked before enforcing the normal limit:

```
bytes32 key =
    _authKey(p.poolId, p.scId, p.originCentrifugeId, p.sender, p.receiver, p.amount);

if (authorizations[key] > 0) {
    authorizations[key]--;
    return;
}

circuitBreakerGuard.tally(
    keccak256(abi.encode(p.poolId, p.scId, p.originCentrifugeId)),
    p.amount,
    l.rateMax,
```

```
    l.rateWindow
);
```

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/283>.

BurraSec: Fix looks good. Authorized transfers now skip the rate-limit tally entirely.

7.1.5 A nonzero-delay StdManifest blocks OracleValuation feeder updates

Target: StdManifest.sol, OracleValuation.sol, Hub.sol

Severity:

- Impact: Medium
- Likelihood: Medium

Description: OracleValuation._setPrice stores the price and, in the same transaction, calls the _protected Hub.updateHoldingValue, so a feeder push lands only if that hub call passes the pool's manifest. Under a standard StdManifest with nonzero delay, with onchainAccounting, updateHoldingValue is pinned to the NAVManager, so the classifier reverts OnchainAccountingOnly for the OracleValuation caller and it cannot even be pre-authorized; otherwise it is deny-by-default delay and reverts Unauthorized absent a matured authorization. The nested revert rolls back the price write, so the feeder commits nothing. Every manager the manifest admits synchronously is bounded (SimplePriceManager by _checkSharePrice, BRM by _checkRequestPrice), but OracleValuation.setPrice is unbounded (quorum 1, any price), so the manifest has no bounded in-policy path for holding valuation. The failure is loud to the operator (the feeder tx reverts) but silent to reads, since getPrice and Hub.pricePoolPerAsset keep returning the last stored price. Recovery is a governance manifest change, which is escalation-timelocked, or a Root spell, while pausing bridges the window.

Recommended Mitigation: Add a price-bounded in-policy classification for updateHoldingValue from a registered valuation contract, mirroring _checkRequestPrice / _checkSharePrice, so an oracle push is in policy up to a deviation bound and out of policy beyond it.

Centrifuge: Fixed in [centrifuge/protocol-internal#283](https://github.com/centrifuge/protocol-internal/pull/283)

BurraSec: Fix looks good. Calls to updateHoldingValue from the configured OracleValuation are now in policy under both accounting modes, while other oracle calls and feeder-management operations remain delayed.

7.1.6 Global-group votes leak into a pool's group, letting one relayer in each take over the pool

Target: MultiAdapter.sol

Severity:

- Impact: High
- Likelihood: Low

Description: MultiAdapter only accepts a message from another chain after enough relayers (adapters) confirm it, for example 2 out of 3. There are two groups of relayers: one global (default) group, and an optional group that each pool can set for itself. When a relayer confirms a message, its vote goes into a counter that is identified only by the message itself. The counter does not record which group the vote came from.

The message that sets a pool's relayers is a special case. Before pool P has its own group, this message is checked against the global group. After pool P has its own group, the same message is checked against pool P's group. Since it is the same message, both checks use the same vote counter, and votes are never reset between them. So one vote from the global group and one vote from pool P's group get added together in a single counter.

An attacker with **one adapter in the global set and one in P's set** (each below its own threshold) can take over P:

1. Before P is configured, the global adapter votes on a malicious SetPoolAdapters(P, attackerAdapters, threshold=1). It routes to the global set. The global threshold is not met, so nothing runs, but the vote stays.

2. P is later configured normally.
3. The pool adapter votes once on the same message, now routing to P. The leaked global vote plus this pool vote reach P's threshold, so it executes and swaps P's adapters for the attacker's.

POC:

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity 0.8.28;

import {MultiAdapterTest, POOL_A, POOL_0} from "./MultiAdapter.t.sol";

import {IAdapter} from "../../src/core/messaging/interfaces/IAdapter.sol";

contract PoC_H1_CrossSetVoteLeak is MultiAdapterTest {
    function test_PoC_H1_globalVoteLeaksIntoPoolThreshold() public {
        uint16 cid = REMOTE_CENT_ID;

        // Attacker controls exactly ONE adapter in each set. Both sets are 2-of-3, so neither
        // compromise alone (nor both, under the intended model) should forge anything.
        address attackerGlobal = makeAddr("attackerGlobalAdapter");
        address attackerPool = makeAddr("attackerPoolAdapter");

        // Global set (pool 0), 2-of-3, attacker at index 0 => id 1 => vote slot 0.
        IAdapter[] memory globalSet = new IAdapter[](3);
        globalSet[0] = IAdapter(attackerGlobal);
        globalSet[1] = IAdapter(makeAddr("honestGlobal2"));
        globalSet[2] = IAdapter(makeAddr("honestGlobal3"));
        multiAdapter.setAdapters(cid, POOL_0, globalSet, 2); // global session 1

        // The malicious message: a SetPoolAdapters targeting POOL_A (mock routes 0x04-prefixed msgs to
        // POOL_A). Wrapped with session id 1 - both the global set and a fresh pool bootstrap at
        // session 1, so the attacker references session 1 in the prefix.
        bytes memory wrapped = _wrap(1, SET_POOL_ADAPTERS_MSG);
        bytes32 boxKey = keccak256(wrapped);

        // Step 1: POOL_A is NOT configured yet -> the message routes to the GLOBAL set.
        // The attacker's GLOBAL adapter casts one vote.
        assertEq(multiAdapter.activeSessionId(cid, POOL_A), 0, "precondition: POOL_A unconfigured");

        vm.prank(attackerGlobal);
        multiAdapter.handle(cid, wrapped);

        // Global threshold (2) is not reached with a single vote, so nothing is forwarded...
        assertEq(gateway.count(cid), 0, "nothing forwarded after 1 global vote");
        // but the vote is recorded in the shared box at slot 0 and is NEVER cleared.
        int16[8] memory afterStep1 = multiAdapter.votes(cid, boxKey);
        assertEq(afterStep1[0], 1, "leaked global vote sits in slot 0");
        assertEq(afterStep1[1], 0);

        // Step 2: POOL_A is bootstrapped legitimately (2-of-3). Attacker is one of its adapters, placed
        // at index 1 => id 2 => vote slot 1 (a DIFFERENT slot from the global vote).
        IAdapter[] memory poolSet = new IAdapter[](3);
        poolSet[0] = IAdapter(makeAddr("honestPool1"));
        poolSet[1] = IAdapter(attackerPool);
        poolSet[2] = IAdapter(makeAddr("honestPool3"));
        multiAdapter.setAdapters(cid, POOL_A, poolSet, 2); // pool session 1 - does NOT clear_votes

        // Step 3: the SAME message now routes to POOL_A's set. The attacker's POOL adapter casts a
        // SINGLE vote. Combined with the leaked global vote in slot 0, the tally reaches 2.
        vm.prank(attackerPool);
        multiAdapter.handle(cid, wrapped);
    }
}
```

```

// LEAK: the malicious SetPoolAdapters is forwarded to the gateway with only ONE genuine
// pool-set vote - the second "vote" was leaked from the global set.
assertEq(gateway.count(cid), 1, "H-1: message forwarded with only 1 real pool vote");
assertEq(gateway.handled(cid, 0), SET_POOL_ADAPTERS_MSG, "forwarded payload is the attacker's msg");

// a fresh SetPoolAdapters (different bytes -> different box) that receives only ONE
// pool vote and no leaked global vote is NOT forwarded. This isolates the leak as the
// cause: 1 pool vote alone is insufficient; 1 pool vote + 1 leaked global vote is not.
bytes memory freshWrapped = _wrap(1, hex"04ffeeddccbbaa99");
vm.prank(attackerPool);
multiAdapter.handle(cid, freshWrapped);
assertEq(gateway.count(cid), 1, "control: a lone pool vote does NOT forward (count unchanged)");
}
}

```

Recommended Mitigation: Include the routed poolId in the vote tally key so votes cast under different adapter sets can never share a slot - e.g. change `_votes` to `mapping(uint16 centrifugeId => mapping(PoolId poolId => mapping(bytes32 payloadHash => int16[MAX_ADAPTER_COUNT])))` (and pass the resolved poolId through `_vote/_execute`), or equivalently fold poolId into payloadHash. Simply clearing `_votes` on setAdapters is **not** sufficient, because the stale vote lives under the global pool-0 namespace while the pool bootstrap writes under pool P's namespace - the two never overlap for clearing but do overlap for counting.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279>.

BurraSec: poolId is now part of the `_votes` mapping key by being encoded into `voteKey`. This mitigates the vote double-counting bug.

7.1.7 Message types were renumbered, so messages sent before the update are read as the wrong type after it

Target: [MessageLib.sol](#)

Severity:

- Impact: Medium
- Likelihood: Medium

Description: Every cross-chain message starts with a number that says its type. That number is just the position of the type in the `MessageType` enum. In v3.3, `RecoverTokens` (and some reserved gap slots) were removed, so every type after it shifted down by one. A message that was created before the update still carries the old number, and after the update the code reads that same number as a different type.

Deployed v3.1 numbering:

```

enum MessageType {
    _Invalid,          // 0
    ScheduleUpgrade,   // 1
    CancelUpgrade,     // 2
    RecoverTokens,     // 3  <-- removed in v3.3
    RegisterAsset,     // 4
    SetPoolAdapters,   // 5
    NotifyPool,        // 6
    // ...
}

```

Current v3.3 numbering:

```

enum MessageType {
    _Invalid,          // 0
    ScheduleUpgrade,   // 1
    CancelUpgrade,     // 2
    RegisterAsset,     // 3  (was 4)
}

```

```
SetPoolAdapters, // 4 (was 5)
NotifyPool,      // 5 (was 6)
// ...
}
```

The number 5 used to mean `SetPoolAdapters`, but now means `NotifyPool`. So a `SetPoolAdapters` message created before the update is read as a `NotifyPool` after it. This happens for any message that survives the upgrade: messages saved in `failedMessages` or `underpaid` and retried later, and messages sent from a chain that has not upgraded yet to a chain that has. The wrong handler runs. It gets worse in a batch, because the type also decides the message length used to split the batch, so a wrong length makes the code misread the following bytes as more messages.

Recommended Mitigation: Keep the numbers stable across versions. Either put back the reserved gap slots so removed types keep their old positions, or make the rollout safe: upgrade all chains together and clear every stored and in-flight message (`failedMessages`, `underpaid`, `queues`) before the switch, so no old-numbered message is ever read by new code.

Centrifuge: We will communicate with all current issuers on Centrifuge that they block investment requests by unlinking vaults, and then have a time period where no other messages are initiated. When all pending messages are delivered, the spell can be executed. For any failed messages that are pending retry, we will manually inject fixed messages in the spell.

BurraSec: The selected approach will ensure that there won't be any message conflicts after the upgrade and will prevent this issue from happening.

7.1.8 Stale adapter updates can roll back active adapter sets

Target: `src/core/hub/Hub.sol`, `src/core/messaging/MultiAdapter.sol`, `src/core/messaging/MessageProcessor.sol`, `src/core/messaging/libraries/MessageLib.sol`

Severity:

- Impact: High
- Likelihood: Low

Description: Stale `SetPoolAdapters` messages sent under an old adapter session can be delivered after a newer adapter set has been installed and can roll the route back to the old payload's adapter list. `Hub.setAdapters()` intentionally sends the remote `SetPoolAdapters` message before installing the local adapter set, so the update travels over the currently shared session. The serialized `SetPoolAdapters` payload contains only `poolId`, `threshold`, and `adapterList`; it does not include an expected receiver session, nonce, or monotonic configuration version. On the inbound side, `MultiAdapter._resolve()` reads the session id from the wrapped payload and verifies the submitted adapter against that payload session, not against the pool's current `activeSessionId`. Old sessions therefore remain executable until explicitly blocked, as also documented by `IHub.setAdapters()`.

When the old-session `SetPoolAdapters` reaches quorum, `MessageProcessor.handle()` deserializes the old adapter list and calls `multiAdapter.setAdapters(centrifugeId, poolId, adapters, m.threshold)` without checking whether the message was superseded by a later adapter configuration. This increments the receiver's session and makes the stale payload the new active adapter set. The result is a rollback of the route's active adapters to an earlier configuration, even if a non-malicious admin, governance action, failover, or emergency operation had already installed a newer set.

This does not require a malicious admin. The admin may legitimately rotate adapters to remove a compromised, unavailable, or unsafe route; the attack surface is the stale message that was already authorized by the old session and is delivered later. If an old threshold-1 route remains unblocked, the removed adapter or transport can still deliver an old `SetPoolAdapters` payload under its former session and re-activate an obsolete or attacker-favorable adapter set. Threshold-1 routes exist in the checked deployment configuration, for example the Pharos routes in `env/connections/mainnet.json`. The likelihood is low because the clean exploit requires stale delivery/replay, a compromised or unsafe old threshold-1 adapter/transport, or a receiver-side/manual/failover install followed by delivery of the old payload; ordinary old-session message acceptance is an intended liveness feature and can be mitigated operationally by blocking old sessions. However, the current protocol does not make `SetPoolAdapters`

itself freshness-bound, so correctness depends on every legitimate emergency rotation remembering to block all dangerous old sessions before they can deliver stale configuration messages.

Recommended Mitigation: Make adapter-configuration messages freshness-bound. Include an expected receiver session id, a monotonically increasing adapter configuration nonce, or a hash of the current adapter-domain state in `SetPoolAdapters`, and have `MessageProcessor` or `MultiAdapter` reject the message unless it matches the receiver's current expected configuration version. For emergency removals and failovers, block or invalidate superseded sessions atomically with installing the replacement set, or make `setAdapters()` invalidate prior `SetPoolAdapters` messages for the same `(centrifugeId, poolId)`. Add regression tests where session 1 sends `SetPoolAdapters(A)`, session 2 installs `SetPoolAdapters(B)`, and later delivery of the session 1 payload must not make A active again.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279>.

BurraSec: Fix looks good. The newly introduced `targetSessionId` requires each update to match the destination's next session, so old messages cannot roll back the active adapter set.

7.1.9 Delayed Emergency Freezes Let Shareholders Evade Restrictions

Target: `src/manifests/StdManifest.sol`

Severity:

- Impact: High
- Likelihood: Low

Description: V3.3 routes `Hub.updateRestriction()` through `StdManifest`, but the manifest has no selector-specific classification and applies the standard delay to every restriction. This is appropriate for `Unfreeze` and `Member`, which restore or grant rights, but it also delays the strictly tightening `Freeze` action and publicly reveals the exact target and maturity time through `AuthorizationScheduled`.

During this window, a compromised or sanctioned shareholder can escape with its entire position. It can bridge the shares to a new address on another spoke, where the eventual address- and token-specific freeze cannot reach them, or request redemption with a clean attacker-controlled controller. The latter immediately moves the shares into escrow and records the future underlying-asset claim under the clean controller, so subsequently freezing only the original owner does not contain the redemption. Simpler local transfers can likewise move or liquidate the full position before the freeze becomes effective. Managers must discover each new address and schedule another delayed freeze, enabling repeated address or spoke rotation.

The older theory about reordered restriction messages is pre-existing and out of scope; the scoped regression is only V3.3's new delay on emergency freezing.

Recommended Mitigation: Return zero delay only for a fully validated canonical `Freeze` payload. Keep `Unfreeze`, `Member`, malformed payloads, and unknown future variants on the standard delay.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279>.

BurraSec: Fix looks good. Valid `Freeze` updates now execute immediately, while all other restriction updates remain delayed.

7.1.10 Circuit Breaker Replacement Resets Active Rate-Limit Budget

Target: `src/hooks/bridge/BridgeCircuitBreaker.sol`, `src/managers/spoke/guards/CircuitBreakerGuard.sol`, `src/core/hub/Hub.sol`

Severity:

- Impact: High
- Likelihood: Low

Description: `BridgeCircuitBreaker` derives a tally key from `(poolId, scId, originCentrifugeId)`, but `CircuitBreakerGuard` additionally namespaces storage by `msg.sender`. Replacing breaker A with identically configured breaker B therefore gives B an empty tally while A's window remains active. A compromised authenticated origin can exhaust A and, after an authorized hot replacement, spend B's fresh allowance, permitting at most one additional `rateMax` for identical configurations. Medium severity applies only if supported replacements must preserve one continuing per-origin budget. If replacement intentionally begins a fresh policy epoch, or the protocol enforces quiescence until A's window expires, this is Informational. The existing proof demonstrates separate caller namespaces but not the complete Hub-level migration and economic impact.

Recommended Mitigation: Use a stable tally namespace across breaker replacements, migrate active totals and windows, or enforce on-chain quiescence until the replaced breaker's window expires.

Centrifuge: `CircuitBreakerGuard` namespaces tally storage by `msg.sender`, so replacing a breaker resets its window. Pool Managers should expect a fresh window on guard rotation.

BurraSec: Acknowledged

7.1.11 Stale Oracle Retry Can Roll Back a Newer Price

Target: `src/core/messaging/Gateway.sol`, `src/valuations/OracleValuation.sol`, `src/core/hub/HubRegistry.sol`, `src/manifests/StdManifest.sol`, `src/vaults/BatchRequestManager.sol`

Severity:

- Impact: High
- Likelihood: Low

Description: A failed remote oracle update remains permissionlessly retryable, while manifest authorization covers only `Hub.updateHoldingValue(poolId, scId, assetId)` and does not bind the upstream price or message generation. Retrying an old stale-low price P1 can consume authorization intended for a later update and overwrite newer price P2. A compromised pool manager can then approve redemptions at P1; because it matches the rolled-back Hub reference, the BRM deviation guard treats it as in policy, and the payout formula overallocates escrowed assets.

Recommended Mitigation: Bind holding-value authorizations to the intended price, source, feeder, and generation or message hash, and invalidate older failed generations whenever a newer price commits.

Client: Fixed in <https://github.com/centrifuge/protocol-internal/pull/283>

BurraSec: The fix is effective and there is no more stale oracle retry authorization consumption issue.

7.1.12 Ceil-Rounded Deviation Band Permits Zero BRM Price

Target: `src/misc/types/D18.sol`, `src/manifests/StdManifest.sol`, `src/vaults/BatchRequestManager.sol`

Severity:

- Impact: High
- Likelihood: Low

Description: `D18.withinDeviation()` rounds the permitted absolute deviation upward with `ceil(maxDeviation * reference / 1e18)`. At low nonzero reference prices this does not merely admit dust: it can admit a price that is completely outside the configured relative bound. For example, when the committed reference is one raw D18 unit and `maxBrmPriceDeviation` is 1% (1e16), the rounded allowance is one raw unit, so both zero and two pass even though each is 100% away from the reference. `StdManifest._checkRequestPrice()` uses this predicate for BRM issue, revoke, deposit-approval, and redeem-approval prices and classifies a passing call as immediately in policy, without authorization delay or sentinel veto.

A compromised pool manager can use the zero-price special cases to settle an entire epoch for no return. If the committed `pricePoolPerShare` is raw 1, the manager can approve deposits at the honest asset price and then submit `IssueShares` with a share price of zero. `_issueShares()` skips the conversion, advances the epoch, and callbacks zero issued shares; subsequent claims consume the approved deposits while paying users zero shares. The affected assets can be ordinary valuable tokens such as USDC: a large pending epoch can predate the low share price or accumulate from individually valid requests, while no aggregate projected-share cap is checked before approval or zero-price issuance.

The symmetric redemption path applies when the committed `pricePoolPerAsset` is raw 1. The manager can submit `ApproveRedeems` with an asset price of zero and later revoke the approved shares using the honest share price. `_revokeShares()` then records zero payout assets while revoking every approved share, and claims consume users' valuable shares for no payout. In both paths, the astronomical quantity would be the honest conversion output; the malicious zero branch avoids producing that output while still consuming the users' ordinary deposits or shares. The maximum loss is therefore the full approved amount in each affected epoch, potentially repeated across epochs, rather than one raw price unit.

A material instance still requires an honestly committed price in the very-low precision regime, a sufficiently large pending epoch, and compromise of a current pool manager, making likelihood Low. However, no minimum nonzero price invariant prevents this state, and the manifest is specifically intended to contain a compromised manager. The epoch-wide loss gives High impact and Low likelihood, hence Medium severity under the program matrix.

Recommended Mitigation: Enforce the relative bound without rounding the permitted deviation upward. Compare the full-precision products directly, for example `abs(value - reference) * 1e18 <= maxDeviation * reference`; the `uint128` operands make both products safe in `uint256`. Explicitly reject a zero BRM price when the committed reference is nonzero, and add regression tests for references near the D18 precision floor across every BRM action, including `reference = 1` with supplied prices zero, two, and three.

Client: Fixed in <https://github.com/centrifuge/protocol-internal/pull/283>

BurraSec: The above finding has been completely fixed by exact cross-multiplied comparison, preventing dust reference prices from admitting zero BRM prices.

7.1.13 Reordered SetPoolAdapters messages can desynchronize adapter configurations

Target: `Hub.setAdapters`, `MultiAdapter.send / _resolve`, `MultiAdapter._routePoolId`, `MessageLib.routePoolId`

Severity:

- Impact: High
- Likelihood: Low

Description: `Hub.setAdapters` sends `SetPoolAdapters` before applying the new local set so the update travels over an adapter configuration still shared with the destination. `MultiAdapter.send` chooses that carrying configuration through `_routePoolId`: it uses the global set while the target pool is unconfigured and the pool-specific set afterward.

The sender prefixes the message with the carrying set's numeric session id, but not the pool from which that id was taken. On receipt, `_resolve` extracts the id and independently recomputes the routing pool from the destination's current state:

```
PoolId poolId = _routePoolId(centrifugeId, payload);
Adapters memory adapters_ = _activeAdapters[centrifugeId][poolId];
bytes memory wrappedPayload = abi.encodePacked(adapters_.sessionId, payload);
```

```
uint16 sessionId = uint16(bytes2(payload[0:2]));
unwrappedPayload = payload[2:];
PoolId poolId = _routePoolId(centrifugeId, unwrappedPayload);
details = _adapterDetails[centrifugeId][poolId][sessionId][adapter];
```

If the source and destination are at different bootstrap stages, they can resolve the same numeric prefix against different configurations. Global session 1 and pool session 1 are therefore interchangeable, allowing reordered updates to authenticate against the wrong adapter set.

Failure Scenario:

1. Both chains have global adapter session 1, and the destination has no pool-specific adapter set.
2. A manager executes update A. It is sent over global session 1; the source then installs pool configuration A as pool session 1.
3. Before A arrives, the manager executes update B. It is sent over pool session 1; the source then installs pool configuration B as pool session 2.
4. B arrives first. Because the destination pool is still unconfigured, it is checked against global session 1 and, if A and the global set overlap sufficiently, executes and installs B as destination pool session 1.
5. A arrives next. The destination is now configured, so the same numeric prefix is checked against pool session 1 (configuration B). If the global and B sets overlap sufficiently, A also executes and installs A as destination pool session 2.

The source ends on configuration B at pool session 2, while the destination ends on configuration A at the same session id. A threshold-only change using the same adapters satisfies both overlap conditions, so this can leave the destination on a lower threshold after the hub has raised it.

Impact: The hub and destination can remain on different adapter configurations until a later corrective update reaches the destination. This can break cross-chain messaging, preserve an adapter that was meant to be removed, or leave the destination enforcing a weaker threshold than the hub expects.

Recommended Mitigation: Consider adding `targetSessionId` to `SetPoolAdapters` and having the hub use the same derived next session id for the remote message and local installation. `MultiAdapter.setAdapters` should revert unless this id equals the target pool's next session.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279>.

BurraSec: Fix looks good. The newly introduced `targetSessionId` requires each update to match the destination's next session, so updates delivered out of order revert instead of installing mismatched configurations.

7.2 Low Risk

7.2.1 Fixed force-cancel opt-in lets a manager repeatedly wipe an investor's fresh requests

Target: BatchRequestManager.sol, StdManifest.sol

Severity:

- Impact: Low
- Likelihood: Medium

Description: The manifest classifies the request manager's force-cancel actions (`ForceCancelDepositRequest`, `ForceCancelRedeemRequest`) as in-policy, so they run instantly with no timelock and no sentinel-veto window. The batch request manager gates force-cancel solely on a per-investor flag (`allowForceDepositCancel` / `allowForceRedeemCancel`) that is set to true when the investor self-cancels a request and is never reset to false. Once an investor self-cancels any request, a malicious or compromised pool manager can therefore force-cancel every future request that investor submits, instantly and repeatedly, including fresh requests the investor never asked to cancel. Each call wipes the investor's current pending deposit or redeem, a persistent denial of that investor's participation in the given pool, share class, and asset. The cancelled assets or shares are returned to the investor, so this is grieving rather than theft.

Recommended Mitigation: Reset `allowForceDepositCancel` and `allowForceRedeemCancel` to false in the batch request manager once the investor's pending amount fully drains.

Centrifuge: Acknowledged as intended.

BurraSec: Acknowledged

7.2.2 The sentinel-removal guard blocks only self-removal

Target: Supervisor.sol

Severity:

- Impact: Low
- Likelihood: Low

Description: `Supervisor._checkNotSelfRemoval` reverts only when a pending `RemoveSentinel` cancellation targets the caller themselves (`require(payload.toAddress(44) != sender)`), and does not cover `AddSentinel`. Since removing a sentinel is an out-of-policy, sentinel-vetoable `managerCall`. However, any sentinel can cancel a different sentinel's removal, so two or more colluding sentinels are mutually unremovable while retaining unilateral veto over all out-of-policy ops leading to a capture escapable only by a Root/Guardian `deny()` of the Supervisor. Also, any sentinel can veto `AddSentinel`, blocking reinforcement of the set.

Recommended Mitigation: Document a formal root/guardian recovery procedure for replacing a captured supervisor or rotating its sentinel set.

Centrifuge: Acknowledged as intended. Recovery will be written into the operations runbook

BurraSec: Acknowledged

7.2.3 Hyperlane messages to Monad can be underfunded because the adapter uses a chain-independent receive reserve

Target: `HyperlaneAdapter.send`, `HyperlaneAdapter.handle`

Severity:

- Impact: Low
- Likelihood: Medium

Description: `HyperlaneAdapter.send` adds a fixed `RECEIVE_COST` to the destination gas limit in the Hyperlane hook metadata:

```
uint256 public constant RECEIVE_COST = 4000;

bytes memory metadata = _metadata(gasLimit + RECEIVE_COST, refund);
```

`RECEIVE_COST` is meant to cover the work `handle()` does before calling `entrypoint.handle()`:

```
HyperlaneSource memory source = sources[origin];
require(source.addr != address(0) && source.addr == sender.toAddressLeftPadded(), InvalidSource());

entrypoint.handle(source.centrifugeId, body);
```

That reserve is not chain-aware. Monad `reprices` cold account access from 2600 to 10100 gas and cold storage access from 2100 to 8100 gas. This wrapper has at least one cold `SLOAD` (`sources[origin]`) and one cold `CALL` (`entrypoint.handle`). On Monad those two accesses alone cost about 18,200 gas versus 4,700 on Ethereum, so the 4000 reserve falls short by at least 14,200 gas.

Failure Scenario:

1. A message is routed to Monad through `HyperlaneAdapter` with `gasLimit + 4000` in the metadata.
2. On delivery, the `handle()` wrapper's cold accesses cost more than the reserve covers, so the transaction runs out of gas.
3. Hyperlane leaves the message undelivered; it succeeds only when resubmitted with a higher gas limit.

Impact: Hyperlane messages routed to Monad are systematically underfunded, so automated delivery can fail and stall the carried protocol operations until each message is manually retried with more gas.

Recommended Mitigation: Consider making `RECEIVE_COST` configurable per destination chain instead of a single global constant, and apply the same fix to `ChainlinkAdapter`, `LayerZeroAdapter`, and `AxelarAdapter`, which share the pattern. Consider also refactoring `GasService` to allow `BASE_ADAPTER_COST` and message gas limits to be configured on a per-chain basis, similarly to how `FAILURE_GAS_RESERVE` is already handled.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279> and <https://github.com/centrifuge/protocol-internal/pull/308>.

BurraSec: Fix looks good. All four adapters now adjust `RECEIVE_COST` when the destination chain is Monad.

7.2.4 Hub.setAdapters refund callback reentrancy can desync hub and spoke adapters

Target: `Hub.setAdapters`, `MultiAdapter.send`

Severity:

- Impact: Medium
- Likelihood: Low

Description: `Hub.setAdapters` sends the `SetPoolAdapters` message before applying the local set, so the message carries the session id still shared with the destination:

```
function setAdapters(PoolId poolId, uint16 centrifugeId, IAdapter[] memory localAdapters, bytes32[] memory  
→ remoteAdapters, uint8 threshold, address refund) external payable {  
    ...  
    sender.sendSetPoolAdapters{value: msgValue()}(centrifugeId, poolId, remoteAdapters, threshold, refund);  
    multiAdapter.setAdapters(centrifugeId, poolId, localAdapters, threshold);  
}
```

`MultiAdapter.send` stamps the payload with the current session id, dispatches it, then refunds leftover value to the caller-supplied `refund` through an all-gas call. The callback can therefore reenter `Hub.setAdapters` before the outer call applies its local set.

Both the outer and reentrant messages are stamped with session N. The reentrant call installs its local set as N+1, then the outer call resumes and installs its set as N+2. If the spoke applies the outer message before the inner one, its session N+2 holds the inner call's remote adapters while the hub's N+2 holds the outer call's local adapters.

`setAdapters` is out of policy under `StdManifest`, but both calls can be pre-authorized with individually valid mirrored local/remote pairs. Neither looks incorrect during its vote window; the desynchronization only appears when they are combined through the refund callback.

Attack Scenario:

1. A manager pre-authorizes two valid mirrored updates, A and B, for the same `(centrifugeId, poolId)`. A's refund is a manager-controlled EOA.
2. Before execution, the manager gives that EOA code through EIP-7702, turning it into the reentrancy contract.
3. The manager executes A, which sends under session N before the refund callback reenters with B.
4. B also sends under N and installs its local set as N+1; A then resumes and installs its local set as N+2.
5. If the spoke processes A before B, its N+2 configuration is B while the hub's N+2 configuration is A.

Impact: A manager can desync the hub and spoke `MultiAdapters` using two individually valid `setAdapters` authorizations, leaving both sides on different adapters under the same session id and blocking cross-chain messaging for that `(centrifugeId, poolId)` until manual recovery.

Recommended Mitigation: Consider adding `targetSessionId` to `SetPoolAdapters` and having the hub use the same derived next session id for the remote message and local installation. `MultiAdapter.setAdapters` should revert unless this id equals the target pool's next session.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279>.

BurraSec: Fix looks good. The newly introduced `targetSessionId` makes the remote and local updates target the same session. If a reentrant update for the same pool and chain advances the session first, the outer update becomes stale and the entire transaction reverts.

7.2.5 Hub.cancelAuthorization bypasses the manifest allowlist, leaking a governance-DoS primitive to confined managers

Target: `Hub.cancelAuthorization`, `HubRegistry.cancelAuthorization`, `StdManifest`

Severity:

- Impact: Medium
- Likelihood: Low

Description: The manifest allowlist (restricted/allowed) confines a caller to a fixed selector set for a pool. An allowlisted caller may invoke only its listed selectors, and anything else is blocked outright.

```
if (restricted[poolId][caller]) {  
    require(allowed[poolId][caller][selector], CallerNotAllowed());  
}
```

The scheduling side honors this. `Hub.authorize` calls `HubRegistry.authorize`, which runs `manifest.classify` and reverts for a disallowed selector. `cancelAuthorization` does not. It only checks `_requireManager` and deletes the entry by `authId`, never touching the manifest.

```
function cancelAuthorization(PoolId poolId, bytes calldata data) external {  
    _requireManager(poolId);  
    hubRegistry.cancelAuthorization(poolId, msgSender(), data);  
}
```

`cancelAuthorization` is the veto that removes pending out-of-policy authorizations during their delay window. Because it skips the manifest, any registered manager, including one the allowlist has confined to a narrow selector set, can cancel any pending authorization on the pool. Repeated on each re-authorization, this blocks all out-of-policy governance indefinitely (`updateHubManager`, `updateManager`, `setManifest`, `setAdapters`, etc.), including the authorization that would remove the offending caller.

A caller confined to a narrow selector set such as notifications therefore keeps a pool-wide governance-DoS primitive, exactly the blast radius the allowlist is meant to remove for low-trust callers like keepers and price bots. The condition is recoverable, since `HubRegistry.updateManager` is ward-gated, so Root can flip the offending manager off directly through a spell (subject to the timelock delay).

Impact: A confined manager retains a pool-wide governance-DoS primitive the allowlist is meant to remove, blocking all manager-driven out-of-policy actions until Root intervenes.

Recommended Mitigation: Consider routing `cancelAuthorization` through `_protected` and classifying it as delay 0 in `_classify`, so the allowlist confines it while the veto stays instant for non-restricted callers. Note that this makes the sentinel veto (`Supervisor.cancelAuthorization`) sensitive to allowlist configuration: the Supervisor must never be restricted without `cancelAuthorization` allowed.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279>.

BurraSec: Fix looks good. `cancelAuthorization` now passes through manifest enforcement, so confined managers must have the selector explicitly allowlisted. It remains instant for unrestricted callers, preserving the Supervisor veto path.

7.2.6 A stale cross-chain manager update can restore revoked remote authority

Target: Gateway.sol, MessageProcessor.sol, MessageLib.sol, BalanceSheet.sol, MultiAdapter.sol

Severity:

- Impact: Low
- Likelihood: Low

Description: Cross-chain UpdateManager and SetRequestManager messages carry no version or ordering nonce, and the spoke applies them as blind overwrites, so a corrective update cannot establish that it supersedes an earlier authorized message. An authorized UpdateManager(M, true) can pass Gateway authentication but fail during execution and land in Gateway.failedMessages. After M is found compromised and a later UpdateManager(M, false) removes it, that stale grant stays retryable: Gateway.retry is permissionless and reprocesses it without checking the role's current state, restoring M at any caller's chosen time. The same stale-overwrite occurs if an unordered transport delivers the revoke before the grant. Gateway.clearFailedMessage is the only mitigation, but it is not coupled to revocation, must be repeated per stored instance, and cannot cancel an in-transit message, so an operator revoking M must also find and clear every earlier failed grant before a public retry.

Recommended Mitigation: Version mutable cross-chain configuration. Include a monotonically increasing version in UpdateManager for each (poolId, managerKind, manager) slot and in SetRequestManager for each pool, and store the last applied version on the spoke.

Centrifuge: Intended design

BurraSec: Acknowledged

7.2.7 HyperlaneAdapter never checks that a security module (ISM) is set, so it can silently fall back to the Mailbox default

Target: [HyperlaneAdapter.sol](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: Hyperlane verifies inbound messages with a security module (ISM). The adapter exposes its ISM to the Mailbox, but the ISM starts empty and setIsM accepts a zero address with no check. When the ISM is zero, the Mailbox uses its own default ISM instead of the one the pool intended. This is a downgrade, not an open door, because some verification still runs, and the impact is limited since Hyperlane is only one relayer in the group. It matters most for a pool that uses Hyperlane as its only relayer, where the ISM is the sole check.

References:

- Hyperlane default vs custom ISM: <https://docs.hyperlane.xyz/docs/protocol/ISM/modular-security#default-vs-custom-ism>
- Similar bug (Nucleus): https://solodit.cyfrin.io/issues/1-04-risk-of-relying-on-interchainsecuritymodule-pashov-audit-group-none-nucleus_2024-12-14-markdown

Recommended Mitigation: Set the intended ISM at deployment and verify it during wiring, and document that a zero ISM means use the Mailbox default on purpose. Do not enable a low-threshold (for example 1-of-1) Hyperlane-only set before confirming the ISM.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279>

BurraSec: Fixed by adding a zero-address check to ensure the ISM is intentionally configured. A zero ISM is now explicitly treated as using the Mailbox default, preventing accidental misconfiguration during deployment and wiring.

7.2.8 Circuit-breaker authorizations can be reused across destination chains

Target: `src/hooks/bridge/BridgeCircuitBreaker.sol`, `src/core/hub/HubHandler.sol`

Severity:

- Impact: Medium
- Likelihood: Low

Description: `BridgeCircuitBreaker` large-transfer authorizations are not bound to the destination chain. The bridging hook receives `targetCentrifugeId`, but `BridgeCircuitBreaker._authKey()` only hashes `poolId`, `scId`, `originCentrifugeId`, `sender`, `receiver`, and `amount`. Consequently, a one-shot authorization can be consumed by an otherwise matching transfer to a different destination.

If multiple destinations are enabled, a shareholder can use the same pool, share class, receiver, and exact amount on another target. However, the attacker must own and burn the authorized amount, while the minted shares still go to the fixed receiver. They cannot steal another shareholder's shares, redirect the receiver, create unbacked shares, or increase global issuance.

The strongest supported impact is costly griefing: consuming the authorization can make an intended remote transfer fail after its source shares were burned, requiring reauthorization and retry. A more serious custody or financial impact would require unproven deployment-specific conditions, such as the receiver being attacker-controlled on the alternate chain. Therefore, Medium impact with Low likelihood results in Low severity.

`HubHandler` passing `sender: bytes32(0)` is a separately known and explicitly out-of-scope issue and is not part of this finding's severity.

Recommended Mitigation: Include `targetCentrifugeId` in the authorization and cancellation payloads, events, and `_authKey()`. Add a test proving an authorization cannot be consumed on another destination. Address sender propagation separately.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/283>.

BurraSec: Fix looks good. `BridgeCircuitBreaker._authKey` now folds both `originCentrifugeId` and `targetCentrifugeId` into the authorization key.

7.3 Informational

7.3.1 Superseded adapter sessions authenticate indefinitely

Target: MultiAdapter.sol

Severity:

- Impact: High
- Likelihood: Low

Description: MultiAdapter._resolve authenticates an inbound vote against the session id carried in the message prefix, not the pool's active session: it looks up _adapterDetails[centrifugeId][poolId][sessionId][adapter] and accepts whenever details.id != 0, never checking sessionId == activeSessionId. setAdapters bumps the active session and installs new records on top but never clears the outgoing session; only blockSession does. So every set the pool has ever used stays a valid inbound authenticator under its own session id, indefinitely, until that session is explicitly blocked.

Old-session survival is deliberate, but the implementation keeps every past session forever. That gap makes a compromised-adapter incident persistent: rotating a compromised adapter out with setAdapters does not revoke it, so it still reaches quorum under its stale session and forges inbound messages to gateway.handle (share issuance, price, restrictions), yielding spoke-side control. For a threshold-1 pool a single compromised adapter suffices and the rotation is inert. For threshold >= 2, votes are session-scoped (the payload hash includes the session prefix), so the attacker needs threshold-many adapters that co-existed in one old session. The correct containment (blockSession on every session the adapter belonged to) is neither enforced nor signalled by setAdapters, so an operator who rotates via setAdapters alone believes the adapter is removed when it is not.

Recommended Mitigation: Bound retention instead of removing it: have _resolve reject any session older than a per-pool floor, so the previous session stays valid for in-flight drain but a rotated-out adapter's authority expires after one further rotation. At minimum, document that setAdapters does not revoke and that containment requires blockSession.

Centrifuge: Comment added at [centrifuge/protocol-internal#294](https://github.com/centrifuge/protocol-internal/pull/294)

BurraSec: Documentation added

7.3.2 MultiAdapter.setAdapters accepts threshold == 0, silently disabling inbound adapter consensus

Target: MultiAdapter.sol, AdapterFailover.sol

Severity:

- Impact: Low
- Likelihood: Low

Description: MultiAdapter.setAdapters checks only threshold <= adapters.length, so a nonempty set with threshold == 0 is accepted. execute() then passes its quorum check without a recorded vote. This is a fail-silent configuration that downgrades an intended M-of-N set to one configured adapter. Adapter-Failover.initiateFailover already rejects the same zero threshold, leaving the shared primitive inconsistent.

Recommended Mitigation: Require a nonempty live adapter set to use threshold >= 1.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/270>.

BurraSec: Fix looks good. setAdapters now rejects threshold == 0 on a non-empty set.

7.3.3 Reinstalling a prior manifest revives its unexpired authorizations without a fresh veto window

Target: HubRegistry.sol, Hub.sol, StdManifest.sol

Severity:

- Impact: Low
- Likelihood: Low

Description: HubRegistry keys each authorization as `keccak256(poolId, manifest, data)`, but `setManifest` only overwrites `manifest[poolId]`; it never deletes the old manifest's `authorizedAfter` entries. A swap therefore invalidates old authorizations only while a different address is installed. If the pool rolls back to the exact prior manifest address after being swapped for a newer manifest while there are authorisations pending, `authId` resolves to the original key again and any authorization still within `[validAfter, validAfter + expiry]` becomes executable immediately.

Recommended Mitigation: Add a monotonically increasing per-pool manifest epoch, increment it on every successful `setManifest`, and include it in `authId`. This preserves rollback to a known manifest address while ensuring authorizations from any prior installation can never become reachable again. Or, add documentation on this case.

Centrifuge: Fixed in [centrifuge/protocol-internal#270](#).

BurraSec: Fix looks good. Added a per-pool manifest nonce to the authorization ID which increments on every manifest change, preventing old authorizations from becoming reachable when a prior manifest is reinstalled.

7.3.4 StdManifest does not check the timelock values, so a bad config can remove the delay that protects setManifest

Target: [StdManifest.sol](#)

Severity:

- Impact: Info
- Likelihood: Info

Description: The manifest is the rule set that decides which manager actions run right away and which must wait a timelock and can be vetoed. Replacing the manifest is meant to use the longest delay (`escalation`), so only a ward can do it instantly and a manager has to wait. The constructor never checks the delay values. If a manifest is deployed with `escalation` set to 0, replacing the manifest counts as allowed now, so any manager can swap the manifest in one transaction with no delay and no veto, then install a permissive manifest that lets them do anything. Setting the normal `delay` to 0 has the same effect for every other guarded action.

Recommended Mitigation: Validate the timelock values in the constructor, for example `require(delay > 0 && escalation >= delay)`, and apply the same check in the factory and config.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279>.

BurraSec: Fix looks good. `StdHubManifest` constructor now requires `delay > 0 && escalation > delay && expiry > 0` (`InvalidConfig`).

7.3.5 Adapter Failover Timelock May Expire Before Hub Veto Arrives

Target: `src/managers/adapters/AdapterFailover.sol`, `src/managers/adapters/interfaces/IAAdapterFailover.sol`, `src/manifests/StdManifest.sol`, `script/FullDeployer.s.sol`

Severity:

- Impact: Low
- Likelihood: Low

Description: The Hub veto for a pending `AdapterFailover` is delayed by the pool's `StdManifest`, so an unsafe configuration where `manifest.delay + relay margin >= failover.timelock` would let a compromised steward execute before the veto can arrive. However, `FullDeployer` only proves that the failover timelock is 48 hours; it does not configure or prove an enabled pool with a matching 48-hour manifest delay. The interface also explicitly assigns this timing margin to deployment configuration. Without evidence of a supported unsafe configuration, this is an Informational configuration warning rather than a demonstrated vulnerability.

Recommended Mitigation: Validate on deployment that `failoverTimelock` exceeds the maximum manifest delay plus a conservative detection and relay margin. Escalate severity only if an enabled supported configuration violates this invariant.

Centrifuge: This requirement is already documented in `IAAdapterFailover.sol`: <https://github.com/centrifuge/protocol-internal/blob/main/src/managers/adapters/interfaces/IAAdapterFailover.sol#L23-L26>.

BurraSec: Acknowledged

7.3.6 Initial Managers Can Bypass Hub Manifest Protections

Target: `src/core/hub/Hub.sol`, `src/core/hub/HubRegistry.sol`

Severity:

- Impact: Info
- Likelihood: Info

Description: `HubRegistry.registerPool()` grants the initial manager while leaving the pool manifest unset, and `Hub._protected()` enforces policy only when the manifest is nonzero. During this bootstrap window, the initial manager can call protected Hub methods without manifest delay, payload restrictions, or sentinel veto. A compromised or externally operated initial manager can install a malicious manifest, grant persistent manager roles, alter adapters or request managers, and dispatch unrestricted manager calls. This is not permissionless: exploitation requires selection or compromise of the initial manager before governance installs the intended manifest.

Recommended Mitigation: Install the initial manifest atomically with pool registration before granting operational managers. If zero-manifest bootstrap must remain possible, restrict protected actions in that state to governance wards or an approved manifest factory.

Centrifuge: Intended: the policy layer is opt-in, so a manifest-less pool has an unrestricted manager by design.

BurraSec: Acknowledged

7.3.7 ISO Holdings Accept Updates From Unauthorized Sources

Target: `src/core/types/AssetId.sol`, `src/core/messaging/libraries/MessageLib.sol`, `src/core/messaging/Gateway.sol`, `src/core/messaging/MessageProcessor.sol`, `src/core/hub/HubHandler.sol`, `src/core/hub/Holdings.sol`, `src/core/hub/Hub.sol`, `src/hooks/accounting/NAVManager.sol`

Severity:

- Impact: Low
- Likelihood: Low

Description: ISO-4217 assets bypass the V3.3 source-origin check for `UpdateHoldingAmount` because the same value, zero, represents both an ISO asset's home domain and an unrestricted message source. `AssetId.centrifugeId()` returns the upper 16 bits of an asset ID, while `newAssetId(uint32 isoCode)` stores the ISO code only in the low bits. Consequently, globally registered ISO assets such as USD (`AssetId(840)`) and EUR (`AssetId(978)`) have `centrifugeId() == 0`.

`MessageLib.messageSourceCentrifugeId()` classifies `UpdateHoldingAmount` according to the embedded asset's `centrifugeId`. `Gateway.handle()` then accepts a message when the classified source is either zero or equal to the authenticated transport source. An `UpdateHoldingAmount` for an ISO asset therefore passes the source check from every chain whose adapter quorum is registered for the target pool, whereas the same wrong-source message for a chain-homed asset reverts with `SourceMismatch`.

Consider a pool whose manager has initialized USD as an accounting holding. Initialization is a protected Hub operation, and `NAVManager` permits initializing assets according to their encoded home, including domain zero after that network is initialized. If a different pool-registered spoke chain or its adapter quorum is compromised, the attacker can authenticate an `UpdateHoldingAmount` containing that pool, share class, the existing USD asset ID, an inflated amount, and an attacker-selected `pricePoolPerAsset`. The attacker supplies the current snapshot nonce for its own authenticated source, so the per-source snapshot state orders that source's messages but does not bind it to the asset's authoritative reporter.

`MessageProcessor` forwards the authenticated source but does not compare it with the asset again. `HubHandler.updateHoldingAmount()` applies the amount and supplied price to `Holdings`, whose storage key is only (`poolId`, `shareClassId`, `assetId`), not the reporting `centrifugeId`. Because the ISO holding was already initialized by the protected pool-manager path, `HubHandler` also calls `Hub.updateAccountingAmount()` and journals the forged value. The proven result is unauthorized corruption of the pool/share-class ISO holding's `assetAmount`, `assetAmountValue`, and associated domain-zero accounting entries. Using `isSnapshot = false` is sufficient to commit this corruption without depending on a successful NAV callback.

The corrupted value does not automatically propagate into the shipped `NAVManager/SimplePriceManager` share-price path. `NAVManager` initializes an ISO holding's equity, gain, loss, and liability accounts under `assetId.centrifugeId() == 0`, while `HubHandler` records the snapshot under the authenticated nonzero transport source. A forged message from source C therefore invokes `netAssetValue(poolId, C)`, which excludes the forged domain-zero journal. A later `Hub.updateHoldingValue()` targets snapshot domain zero, but normal local and remote messages do not establish a source-zero snapshot. No standard production path has been demonstrated that makes the corrupted domain-zero value reach PPS, issuance, redemption, or vault settlement.

The prerequisite is a compromised spoke chain or adapter quorum already registered for the affected pool, plus the unusual configuration of an initialized ISO holding, so likelihood is Low. This prerequisite is still relevant to V3.3 because the new origin check explicitly aims to contain compromised-spoke holding corruption. Nevertheless, the demonstrated impact is limited to pool-scoped, recoverable holding and accounting-integrity damage; no theft, unbacked minting, incorrect redemption, or cross-pool impact is proven. Medium impact and Low likelihood therefore produce Low severity. The issue should be upgraded to Medium only if an end-to-end proof shows a normal, non-ward-dependent path from the corrupted domain-zero accounting state into PPS or settlement.

Recommended Mitigation: Do not overload source value zero to mean both a legitimate asset domain and "any source." Make source classification return an explicit restriction flag together with the required `centrifugeId`. Because ISO assets do not encode a unique reporting chain, configure an authoritative source per (`poolId`, `shareClassId`, `assetId`) holding and validate `UpdateHoldingAmount` against it; until such a source exists, reject ISO asset IDs for this message type. Add an integration test that initializes an ISO holding, submits an update from a different pool-registered source, and expects `SourceMismatch`. Retain a positive control for the configured

reporter and a negative control showing that a wrong-source chain-homed asset is also rejected. If Medium severity is sought, the test must additionally demonstrate that the forged value reaches SimplePriceManager, changes PPS, and affects issuance, redemption, or settlement through a normal configuration.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/285>

BurraSec: Fix looks good, it fully fixes the above issue for fresh deployments.

7.3.8 Stale Steward Updates Can Restore Revoked Failover Authority

Target: `src/core/hub/Hub.sol`, `src/core/messaging/MessageProcessor.sol`, `src/core/utils/Envoy.sol`, `src/managers/adapters/AdapterFailover.sol`, `src/core/messaging/MultiAdapter.sol`

Severity:

- Impact: Info
- Likelihood: Info

Description: V3.3 introduces AdapterFailover stewardship updates through remote ManagerCall, but neither the message nor `steward[poolId][who]` includes a nonce or configuration generation. An older UpdateSteward(S, true) can therefore arrive after a newer revoke and restore S. The retired or compromised steward can then initiate failover to an attacker-selected adapter set; after the timelock, execution replaces the pool's messaging trust root. This can expose the affected pool to arbitrary authenticated pool messages. Exploitation requires reversed delivery, control of the retired steward, an enabled failover integration, and failure of the Hub veto path, resulting in Medium overall severity. This is distinct from M15 because it corrupts AdapterFailover.steward through ManagerCall and must traverse the failover lifecycle.

Recommended Mitigation: Include a monotonically increasing generation in each steward update and reject any generation that is not strictly newer for (destination, poolId, steward). Grants and revokes must advance the same counter.

Centrifuge: Given the low cadence of such messages, this is easy for the manager to ensure it doesn't happen.

BurraSec: Acknowledged

7.3.9 Wrapped MultiAdapter session ids can revive orphaned votes

Target: `MultiAdapter.sol`

Severity:

- Impact: High
- Likelihood: Very Low

Description: `setAdapters` "resets" pending votes by incrementing the `uint16` session id, wrapping from `type(uint16).max` back to 1:

```
uint16 sessionId;
unchecked {
    sessionId = activeSessionId[centrifugeId][poolId] + 1;
}
if (sessionId == 0) sessionId = 1;
```

Votes are keyed by `payloadHash = keccak256(sessionId || payload)`, so bumping the session id only makes future payloads hash differently. The old vote counters in `_votes` are orphaned, never deleted. Because the session id is only 16 bits, it does not uniquely identify a configuration: after 65,535 reconfigurations the counter wraps back to a previously used id `N`, and any votes cast under the old session `N` become live again under whatever configuration now occupies `N`.

This breaks the assumption that incrementing the session id invalidates pending votes. A vote left below threshold under one configuration can later count toward consensus under a different one, most concerningly when the wrapped-to configuration has a lower threshold, so fewer live votes are needed to reach it. Since `execute()`

forwards on the positive-slot count alone, a revived vote set can push a message through the gateway without genuine consensus from the current adapters.

Impact: A message rejected under an old configuration can be executed under a later one after the session id wraps, bypassing the intended per-session vote isolation. Likelihood is very low: it requires 65,535 reconfigurations of the same (centrifugeId, poolId) and a wrapped-to configuration weak enough for the stale votes to reach threshold.

Recommended Mitigation: Do not wrap the session id. Revert on exhaustion instead, so an id is never reused and orphaned votes can never be revived:

```
uint16 sessionId = activeSessionId[centrifugeId][poolId] + 1;
require(sessionId != 0, SessionIdExhausted());
```

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279>.

BurraSec: Fix looks good. setAdapters now reverts on sessionId overflow.

7.3.10 Three Hub accounting selectors are unavailable under onchainAccounting

Target: [StdManifest.sol#L220-L231](#), [StdManifest.sol#L163-L167](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: When onchainAccounting is enabled, the nine accounting selectors are restricted to NAVManager. However, NAVManager never calls setAccountMetadata, updateHoldingIsLiability, or setHoldingAccountId, so these functions have no valid caller under this manifest.

Recommended Mitigation: Consider removing these selectors from _isAccountingSelector if pool managers should retain delayed access. Otherwise, document that they are intentionally disabled.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/328>

BurraSec: Fix looks good

7.3.11 HubRegistry.setBridgingHook does not check that the pool exists

Target: [HubRegistry.sol#L219-L222](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: Unlike the other pool setters, setBridgingHook can write to an unregistered pool. Only a direct ward such as Root can reach this case, and Root is trusted, so this is primarily a consistency-hardening issue. Any hook set before registration remains stored when the pool is later registered.

Recommended Mitigation: Consider adding the same exists(poolId_) check used by the other pool setters.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/270>.

BurraSec: Fix looks good. The exists(poolId_) check was added.

7.3.12 Some Hub and HubHandler dependencies cannot be updated

Target: [Hub.sol#L46-L85](#), [HubHandler.sol#L27-L58](#)

Severity:

- Impact: Info
- Likelihood: Info

Description: `Hub.multiAdapter`, `Hub.hubRegistry`, `Hub.accounting`, and `HubHandler.hubRegistry` are non-immutable storage variables, but they are set only in the constructor and are not supported by file. The `IHub` and `IHubHandler` comments also list unsupported file keys and omit supported ones.

Recommended Mitigation: Consider making intentionally fixed dependencies immutable and adding file support for dependencies that must be replaceable. Update the interface comments to match the supported keys.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/283>.

BurraSec: Fix looks good. `Hub.accounting`, `Hub.hubRegistry`, and `HubHandler.hubRegistry` made immutable; `Hub.multiAdapter` added to `file()`. `NatSpec` corrected.

7.3.13 MessageLib.messageLength accepts the invalid message type

Target: [MessageLib.sol#L107-L111](#)

Severity:

- Impact: Info
- Likelihood: Info

Description: `MessageType._Invalid` is 0, but `messageLength()` rejects only values above the largest enum member. It therefore reports an invalid message as a zero-length message. This does not allow the message to be sent or processed because other checks reject it.

Recommended Mitigation: Consider rejecting `MessageType._Invalid` in `messageLength()`

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279>.

BurraSec: Fix looks good. `MessageLib.messageLength` now rejects `MessageType._Invalid`.

7.3.14 OpsGuardian.wire permits the local chain

Target: [OpsGuardian.sol#L69-L72](#)

Severity:

- Impact: Info
- Likelihood: Info

Description: `wire` blocks the mainnet Centrifuge id but not the local Centrifuge id. By contrast, `setAdapters` blocks both. A trusted ops Safe can therefore wire an adapter to the local chain.

Recommended Mitigation: Consider rejecting `multiAdapter.localCentrifugeId()` in `wire` as well.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279>.

BurraSec: Fix looks good. `OpsGuardian.wire` now rejects `multiAdapter.localCentrifugeId()`.

7.3.15 StandbyAdapter does not emit the underlying adapterData

Target: [StandbyAdapter.sol#L69-L71](#), [MultiAdapter.sol#L298-L299](#)

Severity:

- Impact: Low
- Likelihood: Low

Description: `StandbyAdapter.forward` discards the `adapterData` returned by the underlying adapter. Normal `MultiAdapter` sends include this value in `SendPayload`, so standby sends lack the same off-chain delivery meta-data.

Recommended Mitigation: Consider including the returned `adapterData` in the `Forward` event.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/279>

BurraSec: Fix looks good. `adapterData` was added to the `Forward` event.

7.3.16 ManagerCall NatSpec names the wrong trusted-update target

Target: [MessageLib.sol#L577-L579](#)

Severity:

- Impact: Info
- Likelihood: Info

Description: The comment names `ContractUpdater` as the target that unwraps trusted updates and forwards them to `trustedCall`. That role belongs to `ContractUpdaterForwarder`; targeting `ContractUpdater` directly would revert.

Recommended Mitigation: Consider replacing `ContractUpdater` with `ContractUpdaterForwarder` in the comment.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/283>.

BurraSec: Fix looks good. NatSpec corrected.

7.3.17 Supervisor uses stale trusted-call terminology

Target: [Supervisor.sol#L20](#)

Severity:

- Impact: Info
- Likelihood: Info

Description: `Supervisor` is called through `managerCall` but its documentation still refers to `trustedCall`.

Recommended Mitigation: Consider updating the documentation to use `managerCall`.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/283>.

BurraSec: Fix looks good. NatSpec corrected.

7.3.18 INAVManager declares two unused errors

Target: [INAVManager.sol#L69](#), [INAVManager.sol#L72](#)

Severity:

- Impact: Info
- Likelihood: Info

Description: MismatchedEpochs and ExceedsMaxAccounts are not used by NAVManager or any INAVManager consumer.

Recommended Mitigation: Consider removing the unused errors.

Centrifuge: Fixed in <https://github.com/centrifuge/protocol-internal/pull/283>

BurraSec: Fix looks good. Unused errors were removed.